

01 INSTALLATION

Installing vs Code and C compiler

- Code Editor
- Creating a new project
- Explore settings & Extension

Writing your first program.

main() - `int main() { return 0; }`
printf() - `printf("Hello world");`
stdio.h - standard input and output

Run in Terminal

Windows : `gcc hello.c -o hello.exe`
 `hello`
Mac : `gcc hello.c -o hello`
 `./hello`

02 COMMENTS & VARIABLES

Comments

- Single-line: //
- Multi-line: /* ... */
- Syntax
- Keywords vs words

Variables

Understanding the need for variables.

* Introduction to data types:

- int: Stores integers (e.g., 1, 2, 3).
- char: Stores single characters (e.g., 'A').

Declaration and initialisation.

* Naming Conventions:

- Must start with an alphabet or underscore (_).
- Can end with a digit.
- Special characters allowed: \$ and _.
- Cannot use reserved keywords.
- C is case-sensitive.

* Code Readability

- Use camelCase or snake case for variable and function names.

03 DATA TYPES

Data Types

Primitive : int, float, double, char, void

Derived : array, pointers, function

User defined : structure, union, enum

Format Specifier

define the type of data being printed or read

Chart

Escape Sequence

\t - tab, \n - newLine, \b - backspace, \", \', \\, \?

1. SUM OF TWO INTEGERS

Take two integers a & b with some constant values. Print their sum.

*Example 1:

- Input : a = 5, b = 7
- Output : 12

2. ARITHMETIC OPERATIONS

Take two integers a & b with some constant values. Perform arithmetic operations (+, -, *, /) and print with the appropriate message.

*Example 1:

- Input : a = 10, b = 7
- Output : Sum of 10 & 7 is 17
Subtraction of 10 & 7 is 3
Product of 10 & 7 is 70
Division of 10 & 7 is 1

04 READ INPUT

Standard Input/Output

Read :

- Input
 - using scanf()
 - scanf("%formatSpecifier ", &variableName);
 - & -> ampersand or address of
- Output
 - Using printf()
 - Using placeholders with printf()

***PROBLEM** when take char after int

3. SUM & MESSAGE

Take two integers a & b from user. Print their sum with message.

***Example 1:**

- Input: a = 5, b = 7
- Output: The sum of 5 and 7 : 12

4. SWAP TWO NUMBERS

Take two integers a & b from user and print the values after swap their values.

***Example 1:**

- Input: a = 10, b = 7
- Output: After swap : a = 7 and b = 10

05 OPERATORS

Types

- Unary/Binary Operator
- Arithmetic operator
 - + , - , * , / , % , ++ , --
 - int / int results in an integer
- Rational operator
 - < , > , <= , >= , == , !=
 - returns 1 for true, 0 for false
- Logical operator
 - && , || , !
- Assignment operators
 - = , += , -= , *= , /= , %=
- size of operator

06 HEADER FILES

file with a .h extension that contains inbuilt functions
include-

Common header files:

- `stdio.h`
- `stdlib.h`
- `string.h`
- `math.h`
- `floor()`, `ceil()`, `sqrt()`, `pow()`

5 . AREA AND PERIMETER OF RECTANGLE

Take length and breadth of rectangle from user. Calculate and print the area and perimeter.

Example 1:

Input : $a = 5, b = 3$

Output : Area : 15, Perimeter : 16

6 . AREA AND CIRCUMFERENCE OF CIRCLE

Take radius of circle from user. Calculate and print the area and circumference till two decimal place.

Example 1:

Input : radius : 7

Output : Area of circle : 153.94

Circumference of circle: 43.98

7. CALCULATE POWER OF A NUMBER

Take radius of circle from user. Calculate and print the area and circumference till two decimal place.

Example 1:

Input : $x = 5, n = 2$

Output : 25

8. COMPOUND INTEREST

Take principle, rate, and the time period from user. Calculate and print the compound interest.

Example 1:

Input : principle : 1000, rate : 5.0, time = 2

Output : 102.50

07 FLOWCHART

- What is Flowchart
- Why we use?
- Flowchart Symbols
 - START - Oval
 - END - Oval
 - INPUT - Rectangle
 - OUTPUT- Rectangle
 - PROCESS - Parallelogram
 - CONDITION - Diamond
 - ARROW - Directions

08 CONDITIONAL STATEMENTS

Conditional Statements

- **if**
if(condition)...
executes the block if the condition is true.
- **if-else**
if(condition)..else
adds an alternative block if the condition is false.
- **if-else if**
if(condition)..else if(condition)..else
Chain multiple conditions.
- **if ladder**
multiple independent if statements.
if(condition).. if(condition)..

9 . FIND GREATER NUMBER

Take two numbers from user. Find the greater number and print the greater number.

Example 1:

Input : a = 39, b = 69

Output : 69

10 . CHECK EVEN OR ODD

Take a number as input from the user. Check whether the number is even or odd, and then print the result.

Example 1:

Input : n = 23

Output : Odd

11 . VALID VOTER

Accept the name and age of user as input. If the user is 18 or older, print: 'Hello [Name], you are eligible to vote.' Otherwise print: 'Hello [Name], you will be eligible to vote in (18 - age) veears.'

Example 1 :

Input : name = "Polu", age = 19

Output : Hello Polu, you are eligible to vote.

Example 2 :

Input : name = "Polu", age = 16

Output : Hello Polu, you will be eligible to vote in 2 years.

12 . VOWEL OR CONSONANT

Accept a character from user and check if it is a vowel or a consonant. Print the result.

Example 1:

Input: ch = 'u'

Output: Vowel

Example 2:

Input: ch = 'd'

Output: Consonant

13 . LEAP YEAR

Take year as input from the user. Check whether it is a leap year or not, and print the result.

Example 1 :

Input : year = 2020

Output : Leap year

Example 2 :

Input : year = 1900

Output : Not a Leap year

14 . FILM RATING ANALYZER

Write a program that takes a movie rating as input and determines the movie's status based on the following conditions:

Rating	Message to displayed
0.0 to 2.0	Flop
2.1 to 3.4	Semi-hit
3.5 to 4.5	Hit
4.6 to 5.0	Super Hit

Example 1:

Input : 1.8

Output : Flop

Example 2:

Input : 5.0

Output : Super Hit

15 . SHOP DISCOUNT

Write a program that takes an initial amount as input and calculates the final amount after applying a flat discount based on the following conditions:

Amount	Discount
0 - 5000	0%
5001 - 7000	5%
7001 - 9000	10%
more than 9000	20%

Example 1:

Input : 4000

Output : 4000

Example 2:

Input : 10000

Output : 8000

09 ADVANCE CONTROL FLOW

- Ternary Operator

<condition> ? <true_value> : <false_value>

- Type Conversion

Implicit and explicit type casting.

Syntax: <data_t type> <var> = (<data_t type>)
<value>.

- Switch Statements

Allowable data types : int, char

fall-through behaviour

Using Range in switch Case

16 . WEEKDAYS CALCULATOR

Display the weekday corresponding to a number (1–7); print "Invalid Input" if the number is out of range.

Example 1:

Input : 3

Output : Wednesday

17 . VOWEL/ CONSONANTS

Take a single alphabet as input and print "Vowel" if it's a vowel, otherwise print "Consonant".

Example 1:

Input : a

Output : Vowel

Flowchart

Pseudo Code

Dubugging

10 LOOPS

- Why loop are Necessary
- Types of loops
- Entry Control
- Exit Control

for loop

- for (init; condition; increment/decrement).
- Infinite Loops
- break, continue

18 . PRINT NUMBERS FROM 1 TO N

Take a number n as input and print all numbers from 1 to n separated by spaces.

Example 1:

Input : n = 5

Output : 1 2 3 4 5

19 . PRINT NUMBERS FROM N TO 1

Take a number n as input and print all numbers from n to 1 in reverse order, separated by spaces.

Example :

Input : n = 5

Output : 5 4 3 2 1

20 . CALCULATE SUM AND AVERAGE

Take a number n as input, calculate the sum of numbers from 1 to n, and print the sum and average.

Example :

Input : n = 5

Output : Sum : 15, Average : 3.0

21 . FACTORIAL OF N

Take a number n as input and calculate its factorial. Print the result.

Example 1:

Input : $n = 5$

Output : 120

22 . FACTOR OF A NUMBER

Take a number n as input, calculate the factors of a number from 1 to n , and print them.

Example :

Input : $n = 12$

Output : 1 2 3 4 5 6 12

23 . CHECK IF NUMBER IS PRIME

Take a number as input and check whether it is a prime number or not. Print "Yes" if it is prime, otherwise print "No".

Example 1:

Input : $n = 8$

Output : No

11 WHILE LOOP

While Loop

- while(condition)
- Executes its body while the condition is true.
- Suitable when the number of iterations is unknown.
- Infinite Loop

24 . SEPARATE DIGITS AND CALCULATE THEIR SUM

Take a number n as input, separate its digits, and print them in reverse order. Also, calculate and print the sum of the digits.

Example 1:

Input : n = 123

Output : 3 2 1

Sum : 6

25 . REVERSE A NUMBER

Take a number n as input and print its reverse.

Example :

Input : n = 123

Output : 321

26 . CHECK IF A NUMBER IS PALINDROME

Take a number as input and check whether it reads the same backward as forward. Print "Palindrome Number" if it is, otherwise print "Not a Palindrome Number".

Example :

Input : n = 121

Output : Palindrome Number

12 DO-WHILE LOOP

Do-while Loop

- `do{ ... }while(condition);`
- Executes at least once, even if the condition is false.

27 . GUESS GAME

Write a program to generate a random number between 1 and 100 and ask the user to guess it. Display Too High, Too Low, or Correct Guess until the correct number is guessed.

Example :

```
Console: Guess the number: 50
        Too High
        Guess the number: 20
        Too Low
        Guess the number: 35
        Correct Guess
```

13 NESTED LOOP

Nested Loop

A loop inside loop.

28 . RIGHT TRIANGLE START PATTERN

```
*  
* *  
* * *  
* * * *  
* * * * *
```

29 . MIRROR RIGHT TRIANGLE

```
      *  
     * *  
    * * *  
   * * * *  
  * * * * *
```

30 . V STAR PATTERN

N is always greater than 3

```
*           *  
 *         *  
  *       *  
   *     *  
    *   *  
     *  
    *
```

14 FUNCTIONS

Functions

- Need of Functions
- Types of Functions

Syntax :

- Function declaration, function prototype
 - Function definition, implementation
 - Function call
- Return Type
- Function Arguments
- Pass by Value
- Pass by Reference(DTL)

PROJECT - 02

31 . CLI UTILITY TOOLKIT

Create a menu-driven CLI utility using functions. The program should perform the following operations:

- 1 . Find the sum of two numbers
- 2 . Find the minimum of two numbers
- 3 . Find the maximum of two numbers
- 4 . Check whether a number is Even or Odd
- 5 . Find the factorial of a number
- 6 . Exit

Each operation should be implemented using a separate function.

15 RECURSION

Function calling itself.

Base Case and Recursive Case

Recursive vs Iterative Approach

When to choose recursion

32 . PRINT 1 TO N

Example :

Input : $n = 5$

Output : 1 2 3 4 5

33 . PRINT N TO 1

Example :

Input : $n = 5$

Output : 5 4 3 2 1

34 . FACTORIAL OF NUMBER

Example :

Input : $n = 5$

Output : 120

16 ARRAYS

Arrays

- Understanding the need
- Declaration and initialisation
 - `data_type variableName[size];`
 - `data_type variableName[size] = {1, 2, 3, 4, 5};`
 - `data_type variableName[] = {1, 2, 3, 4, 5};`
- Indexing behind the scene
 - $\text{arr}[i] = \text{Base address} + (i * \text{size of the data type})$
- Passing array to function

PROJECT - 03

35 . ARRAY STATISTICS TOOL

Write a program to read size of array and n elements into an array and perform the following operations :

- 1 . Find the SUM
- 2 . Find the AVERAGE
- 3 . Find the MAXIMUM
- 4 . Find the MINIMUM
- 5 . Exit

Use functions for each operation.

36 . REVERSE ARRAY

Reverse the elements of an array and print them in the new order.

Example :

Input: 1 2 3 4 5

Output: 5 4 3 2 1

37 . LEFT ROTATE BY 1

Given an array arr of length n. Rotate all elements of the array to the left by one.

Example :

Input: 1 2 3 4 5

Output: 2 3 4 5 1

38 . LINEAR SEARCH

Search for an element in an array by checking each element one by one. Print the index where the element is found, or indicate if it's not present.

Example :

Input: arr = {5, 12, 34, 56, 78}, key = 34

Output: Element found at index: 2

17 MULTI-DIMENSIONAL ARRAY

Multi D Arrays

- 2D Array, 3D Array
- Declaration and initialisation
 - `data_type variableName[size][size];`
 - `data_type variableName[2][2] = {1, 2, 4, 5};`
 - `data_type variableName[2][2] = {{1, 2}, {4, 5}};`

39 . INPUT AND PRINT A MATRIX

Input a matrix of size row x column and print each element of it.

Example :

Input : mat = {{1, 2, 3}, {4, 5, 6},{7, 8, 9}}

Output : 1 2 3
 4 5 6
 7 8 9

40 . DIAGONAL SUM

Calculate the sum of the diagonal elements of a square matrix and print the result.

Example :

Input : mat = {{1, 2, 3}, {4, 5, 6},{7, 8, 9}}

Output : Sum : 25

18 CHARACTER ARRAY/STRINGS

Strings

- sequence of characters terminated with a null character '\0'.
- String declaration & Initialisation

```
char str[] = "shery";
char str[50] = "shery";
char str[6] = {'s', 'h', 'e', 'r', 'y', '\0'};
char str[] = {'s', 'h', 'e', 'r', 'y', '\0'};
```
- Read a String Input
 - using scanf()
 - using gets()[**deprecate**], puts()
 - using fgets(name, size, file)
 - using scanf - scanf("%[^\n]s", str);
- Common Functions (from string.h)
strlen(), strcpy(), strcmp(), strcat()

41 . ACCEPT STRING AND PRINT

Take a string as input, print the string, and also display its length.

Example :

Input : str = "Hello Shery"

Output : String is "Hello Shery" and its length is 11

42 . CHECK PALINDROME

Check if a string reads the same forward and backward. Print "String is Palindrome" if true, otherwise print "String is not Palindrome".

Example :

Input : s = madam

Output : String is Palindrome

43 . TOGGLE EACH CHARACTER

Convert each lowercase letter to uppercase and each uppercase letter to lowercase in a string. Print the modified string.

Example :

Input : s = "Hello Shery"

Output : hELLO sHERY

19 POINTERS

Pointers

- Variables that stores the memory address of another variable.
- Declaration and Initialisation

```
int *p = &variable_name;
```

 - * --value at address
 - & --address of
 - Format specifier : %p , %u
- Dereferencing a Pointer

```
int value = *p;
```
- Double Pointer
 - Pointer to Pointer
- Passing pointer to Function
 - call by value
 - call by reference

20 STRUCTURE

Structure

User-defined data type.

Group different types into a single type.

Structure Definition & Creating variable

Access & Initialization

- using .(dot) operator
- using Initializer list

Copy Structure

Size of Structures

Passing Structure to Functions

Structure Pointer

Array of Structures

typedef

used to provide existing data types with a new name.

PROJECT - 04

44 . STUDENT RECORDS

Write a program to store details of students using an array of structures.

Each student has:

- Roll number
- Name
- Marks

The program should:

- 1 . Take input for all 5 students
- 2 . Display all student details
- 3 . Find and display the student with highest marks

21 UNION

User-defined data type but unlike structure union members are stored in the same memory location.

- Declaration and initialisation

If we make changes in one member reflect to other member.

creating union variable

Access & Initialisation

Size of union

Struct vs union

22 CONSTANT

What is Constant?

#define(Macro Constant)

- Preprocessor directive
- Replaces value before compilation

Syntax

```
#define PI 3.14
```

Macros

const Keyword

- Declares a variable whose value cannot be changed
- Handled during compilation

23 ENUMERATION

enum

Used to assign names to integral constants

Improves readability of code

Syntax

```
enum day { MON, TUE, WED, THU, FRI };
```

Access Enum

```
enum day d = MON;
```

Default & Custom Values

```
enum status { OK = 1, ERROR = -1 };
```

24 DYNAMIC MEMORY ALLOCATION

Dynamic Memory Allocation

`malloc() : ptr = (cast-type*) malloc(size*byte)`
`(int*) malloc(size*sizeof(int))`

`calloc() : ptr = (cast-type*)calloc(n, element-size)`
`(int*) calloc(size, sizeof(int))`

`free() : free(ptr)`

`realloc() : ptr = realloc(ptr, newSize)`

25 FILE HANDLING

File Handling

- What is File Handling?
- Why use files (store data permanently)
- File Operations

`fopen()`, `fclose()`, `fscanf()` / `fprintf()`, `fgetc()` / `fputc()`,
`fgets()` / `fputs()`,

`fread()` / `fwrite()`

File Modes in `fopen()`'

File Pointers

End of File

File vs Console I/O